



Assignment 2

Aditya Fadhillah (hig708)

TA: Therese Lyngby

Program Analysis and Transformation

Department of Computer Science, May 10, 2025

Program Inversion

1. Janus Run-Length Encoder

After running the Janus `encode` procedure with the initial store `text[10] = {7, 7, 7, 3, 3, 5, 5, 5, 5, 0}`, the final state of the store is:

```
code[20] = {7, 3, 3, 2, 5, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
i = 0
j = 0
text[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

The Janus encoder performs run-length encoding by scanning the array `text` and recording each repeated values into the `code` array. For each run, it stores a pair: the value itself and the number of times it occurs consecutively. During encoding, the values from the input array `text` are replaced by zeros, and the variables `i` and `j` are returned to zero at the end, making the procedure reversible.

2. Run-length encoder in a conventional imperative language

The following is a functionally equivalent run-length encoder written in C#:

```
static void Encode(int[] text, int[] code, ref int i, ref int j)
{
    while (text[i] != 0)
    {
        code[j] = text[i];

        while (text[i] == code[j])
        {
            code[j + 1]++;
            text[i] = 0; // zero out text
            i++;
        }

        j += 2;
    }

    // reset i and j to 0
    while (j != 0)
    {
        j -= 2;
        i -= code[j + 1];
    }
}
```

```
-----
text[] = 0 0 0 0 0 0 0 0 0 0
code[] = 7 3 3 2 5 4 0 0 0 0 0 0 0 0 0 0 0 0
i = 0
j = 0
```

This C# encoder replicates the behavior of the Janus version by ensuring that the `text[]` array is zeroed out during encoding, and by resetting the loop indexes `i` and `j` to zero at the end of execution. The indexes are passed by reference, so their final values are part of the output store.

Because the final values of `text`, `code`, `i`, and `j` match those of the Janus encoder, this implementation achieves functional equivalence.

3. Invert the Janus encoder

```
i j text[10] code[20]

procedure decode
  from i = 0 && j = 0 do
    i += code[j+1]
    j += 2
  until code[j] = 0
  from text[i] = 0 loop
    j -= 2
    from code[j] != text[i] do
      i -= 1
      code[j+1] -= 1
    loop text[i] += code[j]
    until code[j + 1] = 0
    code[j] <=> text[i]
  until i = 0 && j = 0

procedure main
  code[0] += 7
  code[1] += 3
  code[2] += 3
  code[3] += 2
  code[4] += 5
  code[5] += 4
  code[6] += 0
  call decode
```

```
-----

code[20] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
i = 0
j = 0
text[10] = {7, 7, 7, 3, 3, 5, 5, 5, 5, 0}
```

4. Inversion rules used for the inversion of encode

The inversion of the Janus encoder into the decoder is based on the following general program inversion rules:

- The statement `a += b` is inverted using `a -= b`.
- The swap operation `a <=> b` is its own inverse.
- A loop of the form `from e1 do s1 loop s2 until e2` is inverted to `from e2 do I[s1] loop I[s2] until e1`, where `I[s]` denotes the inversion of the statements `s1` and `s2`.
- In the encoder, the forward loop accumulates (`i += 1`), while the reverse loop in the decoder subtracts (`i -= 1`).
- The inversion restores the values in `text[]` using the values and counts stored in `code[]`.
- The symmetry of the condition that resets both `i` and `j` to zero ensures that the program remains reversible.

5. Invert the encoder that you wrote in (2)

The following is a brute force mechanical inversion of the C# encoder. Although this does not produce a correct decoder, it demonstrates how we would systematically attempt to invert each statement according to reversible programming rules.

```
static void Decode(int[] text, int[] code, ref int i, ref int j)
{
    // Invert final loop: reset i and j to 0
    while (j != 20)    // assuming max size of code array
    {
        i += code[j + 1];
        j += 2;
    }

    // Invert encoding loop
    while (text[i] != 0)    // logically inverted loop condition
    {
        j -= 2;

        // Invert inner loop
        while (text[i] == code[j])
        {
            i--;
            // No inverse for text[i] = 0 → information loss
            code[j + 1]--;
        }

        // No inverse for code[j] = text[i] → information loss
    }
}
```

This code reveals the main difficulty: `text[i] = 0` and `code[j] = text[i]` discard original information. Unlike Janus, which never loses information due to strictly reversible operations, conventional languages like C# allow destructive overwrites. Therefore, even though we can mechanically invert increments and decrements, we cannot fully recover the original state. This demonstrates the fundamental challenge of inversion in an irreversible language.

Reversible discrete simulation

6. Simulation for free-falling objects in Janus

a. Janus program

```
t v h tend

procedure fall
    from t = 0 loop
        t += 1
        v += 10
        h -= v
        h += 5
    until t = tend

procedure main
    t += 0
    v += 0
    h += 176
    tend += 3
    call fall
```

b. Manual execution trace (forward run)

Initial store:

t = 0, v = 0, h = 176, tend = 3

Statement	t	v	h	tend
from t = 0 (true)	0	0	176	3
t += 1	1	0	176	3
v += 10	1	10	176	3
h -= v	1	10	166	3
h += 5	1	10	171	3
until t = tend (false)	1	10	171	3
from t = 0 (false)	1	10	171	3
t += 1	2	10	171	3
v += 10	2	20	171	3
h -= v	2	20	151	3
h += 5	2	20	156	3
until t = tend (false)	2	20	156	3
from t = 0 (false)	2	20	156	3
t += 1	3	20	156	3
v += 10	3	30	156	3
h -= v	3	30	126	3
h += 5	3	30	131	3
until t = tend (true)	3	30	131	3

Final store:

t = 3, v = 30, h = 131, tend = 3

At each time step, the simulation increases time by 1 second and velocity by 10 m/s, and updating height based on the current velocity and a correction factor.. After 3 seconds, the object have a velocity of 30 m/s and a height of 131 meters.

7. Solve the inverse problem of a falling object

The object is dropped from the top of a tower and hits the ground after 4 seconds with the final velocity of 40 m/s. With Janus it is possible to calculate the height of the tower by running the program backward.

Running the following main after program fall, will allow Janus to run the program backward, without having to manually invert the fall program:

```

t += 4
v += 40
h += 0
tend += 4
uncall fall

```

After running the program in reverse, the resulting store is:

```

h = 80
t = 0
tend = 4
v = 0

```

Therefore, the object was dropped from a height of 80 meters.